Assignment 3 revised

Friday, November 14, 2025 11:18 AM

Name: Tridib Banik

St #: 400514461

Ans. to the Q. No. 1

The proof of the solution being deadlock-free is provided below —

First, some invariants to be defined as follows —

[inv 1] $m(p_1) + m(p_2) + m(p_3) + m(p_4) + m(p_5) = ph1 + ph2 + ph3 + ph4 + ph5$

[inv 2] $|m(p_7)| + |m(p_2)| = 4$

[inv 3] $LF(m(p_4)) + RF(m(p_4)) + m(p_6) = f1 + f2 + f3 + f4 + f4 + f5$

Next 2 cases to be considered as follows —

(i) $m(p_4) + m(p_5) \neq 0$. It means either return_left_fork can be fired or return_right_fork_and_exit_dining_room can be fired.

(ii) $m(p_4) + m(p_5) = 0$. Then from [inv 3] we have:

$$LF(m(p_3)) + m(p_6) = f1 + f2 + f3 + f4 + f4 + f5$$

and from [inv 1] we have:

$$m(p_1) + m(p_2) + m(p_3) = ph1 + ph2 + ph3 + ph4 + ph5.$$

From the definitions of $LF(x)$ and $RF(x)$, we have

$LF(x) \neq RF(x)$ for all $x = ph1, ph2, ph3, ph4, ph5$.

Therefore, if $m(p_3) \neq 0$ then take_right_fork can be fired.

Also, if $m(p_2) \neq 0$ then take_left_fork can be fired.

If $m(p_1) \neq ph1 + ph2 + ph3 + ph4 + ph5$, then either $m(p_3) \neq 0$ on $m(p_2) \neq 0$.

If $m(p_1) = ph1 + ph2 + ph3 + ph4 + ph5$, then

either $m(p_3) \neq 0$ on $m(p_2) \neq 0$.

If $m(p_1) = ph1 + ph2 + ph3 + ph4 + ph5$, then

If $m(p_1) = ph1 + ph2 + ph3 + ph4 + ph5$, we can derive $|m(p_7)| = 4$, $m(p_2) = 0$, and from inv[2], we can derive

which means enter_dining_room can be fined.

∴ The solution is deadlock-free.

Ans. to the Q. No. 2(a)

A model for the gas station with N customers and M pumps is provided below —

```
const N = 3    // Number of customers
const M = 2    // Number of pumps

range C = 1...N
range P = 1...M
range A = 1...2 // Amount of money on Gas

CUSTOMER = (prepay[a:A] → gas[x:A] →
           if (x==a) then CUSTOMER else ERROR).

CASHIER = (customer[c:C].prepay[x:A] →
          start[p][c][x] → CASHIER).

         [x:A] → gas[c][x] → PUMP).
```

```
PUMP = (start[c:C]

DELIVER = (gas[p][c:C][x:A] -> customer[c].gas[x]
             -> DELIVER).
||STATION = (CASHIER || pump[1..M]:PUMP || DELIVER)
        /{pump[i:1....M].start/start[i],
          pump[i:1...M].gas/gas[i]}.

||GASSTATION = (customer[1..N]:CUSTOMER || STATION).
```

Ans. to the Q. No. 2(b)

The specification and check (with N=2, M=3) of a safety property FIFO, ensuring customers are served in the order in which they pay, is provided below —

```
range P = 1..2
property FIFO = (customer[i:P].prepay[A] -> PAID[i]),
    PAID[i:P] = (customer[i].gas[A] -> FIFO
               |customer[j:P].prepay[A] -> PAID[i][j]),
    PAID[i:P][j:P] = (customer[i].gas[A] -> PAID[j]).

||CHECK_FIFO = (GASSTATION || FIFO).
```

This property is expected to be violated.

Ans. to the Q. No. 2(e)

Java implementation for the gas station system with N=2, M=3 is provided below —

```
// GasStationSimulation.java
// Compile: javac GasStationSimulation.java
// Run:     java GasStationSimulation

import java.util.*;
import java.util.concurrent.*;

class GasStation {

    private final int pumpCount = 3;
    private final BlockingQueue<Integer> waitingCustomers = new LinkedBlockingQueue<>();
    private final Pump[] pumps = new Pump[pumpCount];
    private final Cashier cashier = new Cashier();

    public GasStation() {
        for (int i = 0; i < pumpCount; i++) {
            pumps[i] = new Pump(i);
        }
        cashier.start();
        for (Pump p : pumps) p.start();
    }

    public void prepay(int customerId, int amount) {
        System.out.println("Customer " + customerId + " prepaid for amount: " + amount);
        cashier.receivePayment(customerId, amount);
    }

    class Cashier extends Thread {

        private final BlockingQueue<int[]> payments = new LinkedBlockingQueue<>();

        public void receivePayment(int customerId, int amount) {
            payments.add(new int[]{customerId, amount});
        }

        @Override
        public void run() {
            try {
                while (true) {
                    int[] pay = payments.take();
                    int cid = pay[0];
                    int amount = pay[1];

                    waitingCustomers.add(cid);

                    for (Pump p : pumps) {
```

```
                        if (p.tryAssign(cid, amount)) {
                            break;
                        }
                    }
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    class Pump extends Thread {
        private final int pumpId;
        private final BlockingQueue<int[]> assigned = new LinkedBlockingQueue<>();

        public Pump(int pumpId) {
            this.pumpId = pumpId;
        }

        public boolean tryAssign(int customerId, int amount) {
            if (assigned.isEmpty()) {
                assigned.add(new int[]{customerId, amount});
                return true;
            }
            return false;
        }

        @Override
        public void run() {
            try {
                while (true) {
                    int[] job = assigned.take();
                    int customerId = job[0];
                    int amount = job[1];

                    System.out.println("Pump " + pumpId +
                            " delivering " + amount + " units to customer " + customerId);

                    Thread.sleep(1000);

                    Customer.onGasDelivered(customerId, amount);
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

class Customer implements Runnable {

    private static final Map<Integer, Integer> prepaidMap = new ConcurrentHashMap<>();
    private static final Map<Integer, Integer> receivedMap = new ConcurrentHashMap<>();

    private final int id;
    private final GasStation station;
    private final int amount;

    public Customer(int id, GasStation s, int amount) {
        this.id = id;
```

```
        this.station = s;
        this.amount = amount;
    }

    public static void onGasDelivered(int customerId, int delivered) {
        receivedMap.put(customerId, delivered);
        int expected = prepaidMap.get(customerId);

        if (expected == delivered) {
            System.out.println("Customer " + customerId + " satisfied. Gas received correctly.");
        } else {
            System.out.println("ERROR: Customer " + customerId +
                    " prepaid " + expected + " but received " + delivered);
        }
    }

    @Override
    public void run() {
        prepaidMap.put(id, amount);
        station.prepay(id, amount);
    }
}

public class GasStationSimulation {
    public static void main(String[] args) throws InterruptedException {

        GasStation station = new GasStation();

        Thread c1 = new Thread(new Customer(1, station, 1));
        Thread c2 = new Thread(new Customer(2, station, 2));

        c1.start();
        c2.start();

        c1.join();
        c2.join();

        Thread.sleep(3000);
    }
}
```

Ans. to the Q. No. 3(a)

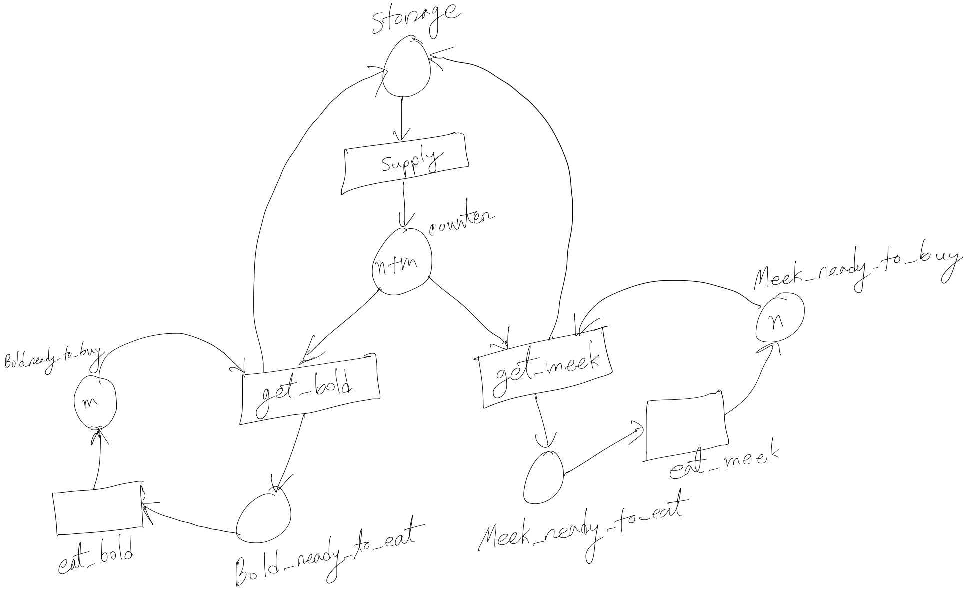
The model of the system with FSP for a fixed population of 2 bold customers and 2 meek customers, assuming there is always cheese available, is provided below—

```
set Bold = {bold[1...2]}
set Meek ={meek[1...2]}
set Customers= {Bold, Meek}

CUSTOMER=(getcheese→ CUSTOMER).
COUNTER= (getcheese → COUNTER).
||CHEESE_COUNTER=(Customers: CUSTOMER || customers :: COUNTER)
```

Ans. to the Q. No. 3(b)

Assuming there is alway cheese available, the modelling of the system with Place/Transition nets is provided below—



Ans. to the Q. No. 3(c)

Including priorities will result in :

```
||CHEESE_COUNTER = (Customers : CUSTOMER
                   || Customers :: COUNTER)>>{Meek.getcheese}.
```

If we use

```
progress BOLD = {Bold.getcheese}
progress MEEK = {Meek.getcheese}
```

Here, Meek.getcheese is starved, since Bold.getcheese will always get executed.

Ans. to the Q. No. 4

The model of the system (with FSP) is provided below—

```
set Bold = {bold[1..2]}
set Meek = {meek[1..2]}
set Customers = {Bold, Meek}
const MT = 4   //maximum ticket number
range T = 1..MT
CUSTOMER = (ticket[t:T]->getcheese[t]-> CUSTOMER).
```

```
TICKET= TICKET[1],
TICKET[t:T] = (ticket[T] -> TICKET[t%MT+1]).

COUNTER = COUNTER[1],
COUNTER[t:T] = (getcheese[t]-> COUNTER[t%MT+1]).

||CHEESE_COUNTER = (Customers: CUSTOMER
                      || Customers :: TICKET
                      || Customers :: COUNTER)
                      >> {Meek.getcheese[T]}.

progress  BOLD = {Bold.getcheese[T]}
progress  MEEK = {Meek.getcheese[T]}
```

Ans. to the Q. No. 5

Translation of the model of the cheese counter from Question 4 into a Java program is provided below -

```
// CheeseCounter.java
// Compile: javac CheeseCounter.java
// Run:     java CheeseCounter
//   Java implementation of Question 4 (Ticket Machine Model)
//   Each customer is a thread that:
//      1. Obtains a ticket
//      2. Waits until its ticket is served
//      3. Gets cheese
//      4. Terminates
// ==============================================================

class TicketMachine {
   private static final int MT = 4;     // max ticket number
   private int next = 0;                // next ticket to issue
```

```java
   // Issue next ticket modulo MT
   public synchronized int getTicket() {
      next = next % MT + 1;
      System.out.println(Thread.currentThread().getName() +
            " received ticket " + next);
      return next;
   }
}

class Counter {
   private static final int MT = 4;     // max ticket number
   private int current = 1;             // ticket currently being served

   // Customer waits until their ticket number is shown
   public synchronized void getCheese(int myTicket) throws InterruptedException {
      while (myTicket != current) {
         wait(); // wait for your turn
      }

      // Serve the customer
      System.out.println("Serving ticket " + myTicket +
            " (" + Thread.currentThread().getName() + ")");

      // Move to next ticket
      current = current % MT + 1;

      // Let waiting customers re-check
      notifyAll();
   }
}

class Customer extends Thread {
   private final TicketMachine ticketMachine;
   private final Counter counter;

   public Customer(String name, TicketMachine tm, Counter c) {
      super(name);
      this.ticketMachine = tm;
      this.counter = c;
   }

   public void run() {
      try {
         // Step 1: take a ticket
         int myTicket = ticketMachine.getTicket();

         // Step 2: wait until the counter displays your ticket
         counter.getCheese(myTicket);

         // Step 3: Finished
         System.out.println(getName() + " received cheese and left.");

      } catch (InterruptedException e) {
         e.printStackTrace();
      }
   }
}

public class CheeseCounter {
   public static void main(String[] args) throws InterruptedException {
```

```
        TicketMachine tm = new TicketMachine();
        Counter counter = new Counter();

        // FSP model has 2 Bold + 2 Meek customers (example)
        Thread c1 = new Customer("Bold1", tm, counter);
        Thread c2 = new Customer("Bold2", tm, counter);
        Thread c3 = new Customer("Meek1", tm, counter);
        Thread c4 = new Customer("Meek2", tm, counter);

        // Start all customers
        c1.start();
        c2.start();
        c3.start();
        c4.start();

        // Wait for all customers to finish
        c1.join();
        c2.join();
        c3.join();
        c4.join();

        System.out.println("\nAll customers have been served.");
    }
}
```

Ans. to the Q. No. 6

A model which can be used to verify that the protocol prevents queue overflow if port are correctly dimensioned is provided below —

// The protocol works by giving the producer N tokens that
// represent available buffer slots in the message queue.
// Before sending a message, the producer must first acquire a token.

```
// Each message sent consumes one token.

// When the consumer receives a message, it sends a token back,

// indicating that one buffer slot has been freed.

// This ensures that the producer can never send more than N messages

// ahead of the consumer, preventing queue overflow.


const N = 3
set M = {msg}
set S = {[M], [M][M]}
PORT = (send[x:M] -> PORT[x]), //empty state, only send permitted
PORT[h:M] = (send[x:M] -> PORT[x][h] //one message queued to port
            | receive[h] -> PORT),
PORT[t:S][h:M] = (send[x:M] -> PORT[x][t][h] //two or more messages queued to port
            | receive[h] -> PORT[t]).
PRODUCER = (empty.receive.token -> dest.send.msg -> PRODUCER).
CONSUMER = SENDBUF[N],
```

SENDBUF[i:1...N] = (empty.send.token →
    if (i==1) then continue
    else SENDBUF[i-1]),

CONTINUE = (dest.receive.msg → empty.send.token → CONTINUE).

||PRODCONS = (PRODUCER || CONSUMER
    || empty : PORT || dest : PORT)
    /{empty.[i:{send, receive}].token/empty[i].msg}.

Ans. to the Q. No. 7(a)(i)

$(\neg p \Rightarrow r)$ is equivalent to $(\neg(\neg p) \vee r) \equiv p \vee r$.

We have $L(s_0) = \{r\}$ so $M, s_0 \models \varphi$.

We have $L(s_2) = \{p, q\}$ so $M, s_2 \models \varphi$.

Ans. to the Q. No. 7(a)(ii)

We have $r \in L(s_0)$ and $r \in L(s_1)$. Moreover, there is an infinite path $s_0 \to s_1 \to s_1 \to s_1 \to \dots$, so $M, s_0 \models EG\, r$. So, we infer $M, s_0 \not\models \neg EG\, r$. Since $r \notin L(s_2) = \{p, q\}$, so $M, s_2 \models \neg EG\, r$ as future includes present.

Ans. to the Q. No. 7(a)(iii)

Since $t \notin L(s_0)$ and $t \notin L(s_2)$, we have

$M, s_0 \not\models E(t \, U \, q)$, and $M, s_2 \not\models E(t \, U \, q)$.

Ans. to the Q. No. 7(a)(iv)

Since $q \in L(s_2)$ and there are infinite paths $s_0 \to s_2 \to, \ldots,$ we have $M, s_0 \models F \, q$. Furthermore, since $q \in L(s_2)$ then $s_0 \models F \, q$ (future includes present).

Ans. to the Q. No. 7 (b)

Statement: 'Event p precedes s and t on all computational paths'.

Negation: 'There exists a path where p does not precede s or does not precede t'.

Ambiguities: Is the case when p never happens allowed? We assume it is not (which means 'yes' for negation). Does 'precede' allow p and s (or p and t) be in the same state? We assume it is not (which means 'yes' for negation).

LTL: $G(F_p \wedge (p \Rightarrow F_s) \wedge (p \Rightarrow F t))$

CTL: $AG(AF_p \wedge AG(p \Rightarrow AF_s) \wedge AG(p \Rightarrow AF t))$

Ans. to the Q. No. 7(c)

Statement: 'Between the events q and r, p is never true but t is always true'.

Ambiguities: Is the case when r or q never happens allowed? We assume that it is not. Also, in the statement, "between" means (the assumption) "closed interval" so p is false in the state that holds q and in the state that holds r.

LTL: $G(F q \wedge F r \wedge (q \Rightarrow (\neg p \, U \, r) \wedge (q \Rightarrow (Ft \, U \, r))))$

CTL: $AG(AF q \wedge AF r) \wedge AG(q \Rightarrow A(\neg p \, U \, r))$

Ans. to the Q. No. 7(d)

CTL: $s \models AG(AF \phi)$

LTL: $s \models G(F \phi)$

Ans. to the Q. No. 7(e)

The process of translating informal requirements into formal specifications is subject to various pitfalls. One of them is simply ambiguity. For example, it is unclear whether "after some finite steps" means "at least one, but finitely many", or whether zero steps are allowed as well. It may also be questionable what "then" exactly means in "... then the system enters...". We chose to solve this problem for the case when zero steps are not admissible, mostly since "followed b" suggest a real state transition to tale place. The LTL formula is the following

$$G(p \Rightarrow XG(\neg q \vee \neg r \; U \; t)),$$

while an equivalent CTL formula is:

$$AG(p \Rightarrow AX\; AG(\neg q \vee A[\neg r \; U \; t])),$$

It tells us that at any state, if p is true, then at any state which one can reach with at least one

is false, or r

state transition from here, either q is false until r becomes true (for all continuations of the computation path). This is clearly the property we intended to model.

Ans. to the Q. No. 7(f)

Statement: 'Between the events q and r, p is never true.'

Ambiguities: Is the case when r or q never happens allowed? Our assumption is that it is not. Also, here, we assume "between" is "closed interval" so p is false in the state that holds q and in the state that holds r.

LTL: $G(F\,q \wedge F\,r \wedge (\neg q \vee (\neg p \,U\, r)))$

CTL: $AG(AF\,q \wedge AF\,r) \wedge AG(q \Rightarrow A(\neg p \,U\, r))$

Ans. to the Q. No. 8

Assume the following atomic predicates that characterize properties of processes:

lpr_i - local processing of reader $i, i = 1, 2,$

lpw_i - local processing of writer $i, i = 1, 2,$

non-critical states

tr_i - reader $i, i = 1, 2$, requests reading,

tw_i - writer $i, i = 1, 2$, requests writing,

trying to enter critical states

r_i — reader $i, i = 1, 2$, is reading,

w_i — writer $i, i = 1, 2$, is writing,

in critical states

The solution restricted to writers only. So, to avoid similar problems,

let's introduce additional boolean variables (or atomic predicates): turn = w1, turn = w2 and turn = r, to indicate that worlds where writer 1 will write (turn = w1), writer 2 will write (turn = w2), or readers (one or both) will read (turn = r).

Next, states can be identified by atomic predicates of the form: (str1, str2, stw1, stw2, turn)

where: $str1 \in \{ lpr_1, tr_1, r_1 \}$, $str2 \in \{ lpr_2, tr_2, r_2 \}$, - status of readers;

$stw1 \in \{ lpw_1, tw_1, w_1 \}$, $stw2 \in \{ lpw_2, tw_2, w_2 \}$, - status of readers;

$turn \in \{ turn = w1, turn = w2, turn = r \}$, - status of turns.

Life of a reader is a simple cycle:

$(lpr_1, *, *, *, *) \rightarrow (tr_1, *, *, *, *) \rightarrow (r_1, *, *, *, *) \rightarrow$ back to start,

similarly, for writers:

$(*, *, lpw_1, *) \rightarrow (*, *, tw_1, *) \rightarrow (*, *, w_1, *) \rightarrow$ back to start.

Not all combinations of atomic predicates are allowed, for example,

$stw1 = w_1 \Rightarrow str1 \neq r_1 \wedge str2 \neq r_2 \wedge stw2 \neq w_2$ or

$str1 = r_1 \Rightarrow stw1 \neq w_1 \wedge stw2 \neq w_2$

Properties are also similar to these for Mutual Exclusion:

Safety in LTL: $G(w_1 \Rightarrow \neg(w_2 \vee r_1 \vee r_2))$,

$G(w_2 \Rightarrow \neg(w_1 \vee r_1 \vee r_2))$,

$G((r_1 \vee r_2) \Rightarrow \neg(w_1 \vee w_2))$

the question asked for either LTL or CTL. So, please consider either LTL or CTL for grading.

in CTL: $AG(w_1 \Rightarrow \neg(w_2 \vee r_1 \vee r_2))$

$AG(w_2 \Rightarrow \neg(w_1 \vee r_1 \vee r_2))$

$AG((r_1 \vee r_2) \Rightarrow \neg(w_1 \vee w_2))$

Liveness in LTL: $G(tr_1 \Rightarrow F\ r_1)$,

$G(tr_2 \Rightarrow F\ r_2)$,

$G(tw_1 \Rightarrow F\ w_1)$,

$G(tw_2 \Rightarrow F\ w_2)$

in CTL: $AG(tr_1 \Rightarrow AF\ r_1)$

$AG(tr_2 \Rightarrow AF\ r_2)$

$AG(tw_1 \Rightarrow AF\ w_1)$

$AG(tw_2 \Rightarrow AF\ w_2)$

the question asked for either LTL or CTL. So, please consider either LTL or CTL for grading.